# Plotting for Exploratory Data Analysis

02 January 2024 13:21

We'll be using the iris dataset.

**Fisher's *Iris* Data**

| Sepal length | Sepal width | Petal length | Petal width | Species |
|---|---|---|---|---|
| 5.1 | 3.5 | 1.4 | 0.2 | *I. setosa* |
| 4.9 | 3.0 | 1.4 | 0.2 | *I. setosa* |
| 4.7 | 3.2 | 1.3 | 0.2 | *I. setosa* |
| 4.6 | 3.1 | 1.5 | 0.2 | *I. setosa* |
| 5.0 | 3.6 | 1.4 | 0.3 | *I. setosa* |
| 5.4 | 3.9 | 1.7 | 0.4 | *I. setosa* |
| 4.6 | 3.4 | 1.4 | 0.3 | *I. setosa* |
| 5.0 | 3.4 | 1.5 | 0.2 | *I. setosa* |
| 4.4 | 2.9 | 1.4 | 0.2 | *I. setosa* |

3 species: Iris setosa, Iris versicolor, Iris virginica

```
print(iris.head())
print(iris.shape)
```

✓ 0.0s

```
   sepal.length  sepal.width  petal.length  petal.width variety
0           5.1          3.5           1.4          0.2  Setosa
1           4.9          3.0           1.4          0.2  Setosa
2           4.7          3.2           1.3          0.2  Setosa
3           4.6          3.1           1.5          0.2  Setosa
4           5.0          3.6           1.4          0.2  Setosa
(150, 5)
```

```
iris["variety"].value_counts()
```

✓ 0.0s

```
Setosa        50
Versicolor    50
Virginica     50
Name: variety, dtype: int64
```

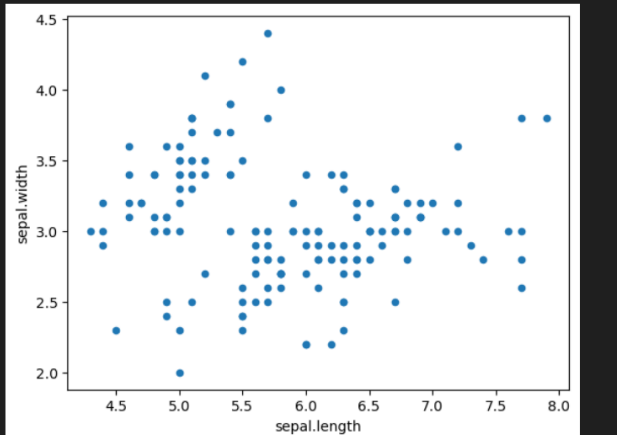
Quite balanced dataset.

---

2-D Scatter Plot

```
iris.plot(kind="scatter", x="sepal.length", y="sepal.width")
plt.show()
```

✓ 0.4s

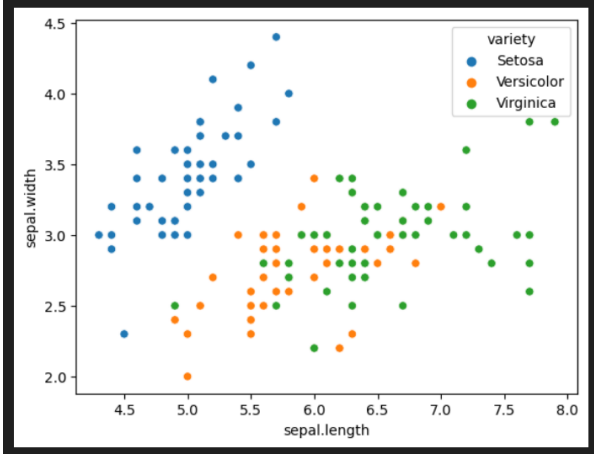


We need to distinguish the species

```
sns.scatterplot(iris, x="sepal.length", y="sepal.width", hue="variety")
```

✓ 0.8s

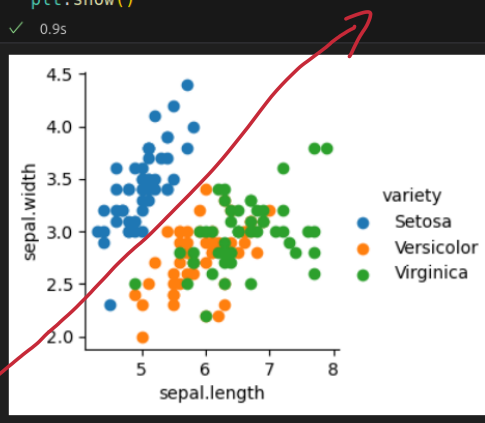
```
<AxesSubplot: xlabel='sepal.length', ylabel='sepal.width'>
```



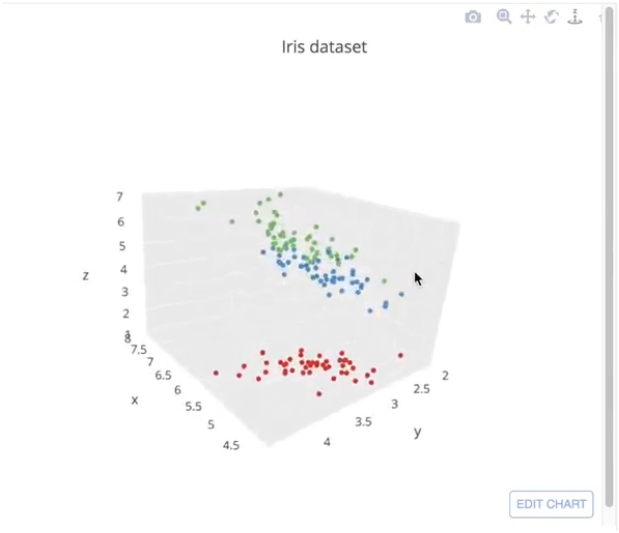

Alternative code:

```
sns.FacetGrid(iris, hue="variety", )\
    .map(plt.scatter, "sepal.length", "sepal.width")\
    .add_legend()
plt.show()
```

✓ 0.9s




Setosa is quite linearly separable from the others using sepal length and width.
A 3-D plot, by adding more features, can make it even better.



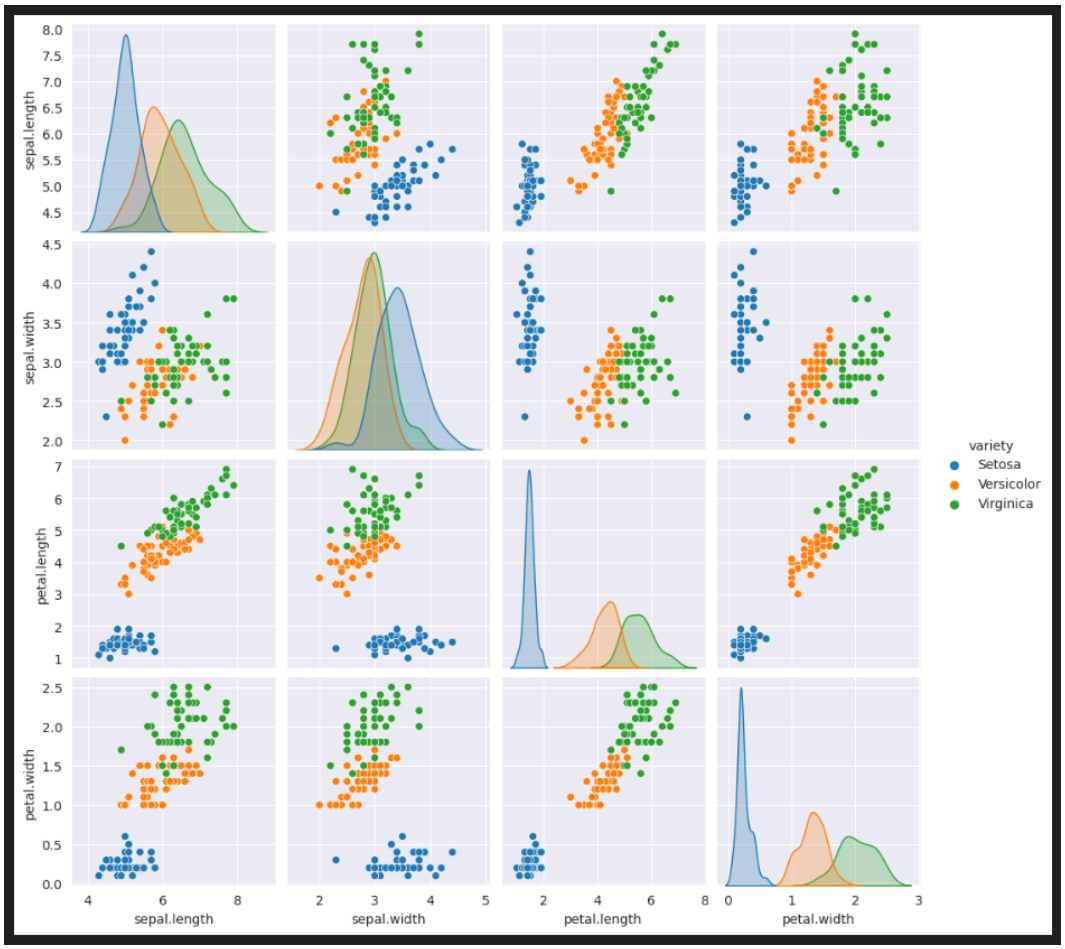
We obviously cannot visualize more than 3 dimensions directly.
So we'll use pair plots.

---

Pair Plots

Pair plots plot features in pairs.
All possible pairs are considered.

```
sns.set_style("darkgrid")
sns.pairplot(iris, hue="variety")
```




Notice that the plot petal.length vs petal.width gives a much better visual distinguishing between the species.

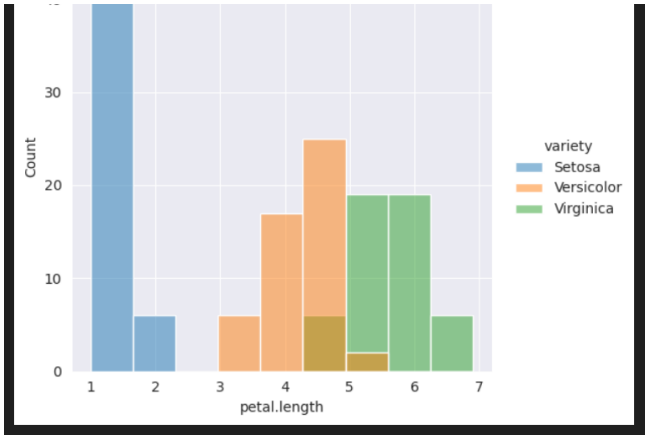
Limitations:
Here we have 4 features, so we need to study 4C2 plots (ignoring the diagonal and upper triangular)
Say for 100 features, we'll need to study a looot of plots.

---

Histogram, PDF, CDF

```
sns.displot(iris, x="petal.length", hue="variety")
✓ 1.6s
<seaborn.axisgrid.FacetGrid at 0x7f3c1833b6d0>
```

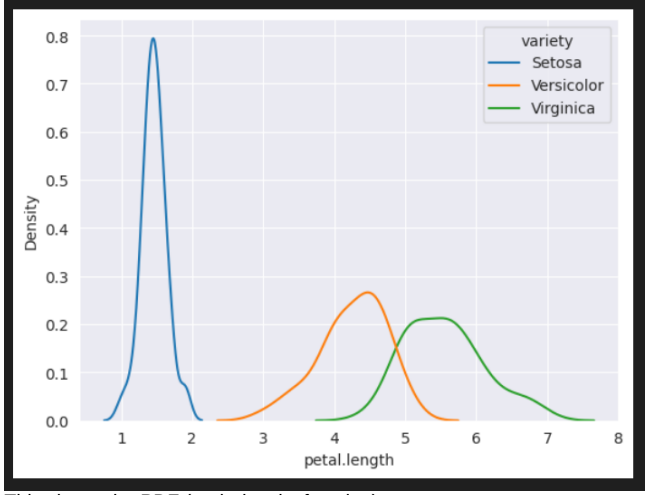





```
sns.kdeplot(iris, x="petal.length", hue="variety")
```

✓ 0.2s

```
<AxesSubplot: xlabel='petal.length', ylabel='Density'>
```




This shows the PDF (prob density function)

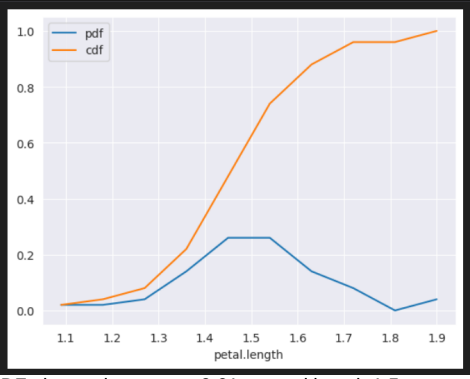
Cumulative Distribution Function

```
counts, bin_edges = np.histogram(iris_setosa["petal.length"], bins=10, density=True)

pdf = counts/sum(counts)
cdf = np.cumsum(pdf)

plt.plot(bin_edges[1:], pdf, label="pdf")
plt.plot(bin_edges[1:], cdf, label="cdf")
plt.xlabel("petal.length")
plt.legend()
plt.show()
```

✓ 0.2s




CDF gives value approx 0.61 at petal length 1.5
So there are 61% of setosa flowers that have petal length <= 1.5
PDF gives value say 0.25.
So 25% of setosa flowers have petal length = 1.5
All values lie between 0 & 1. CDF covers all values obv.

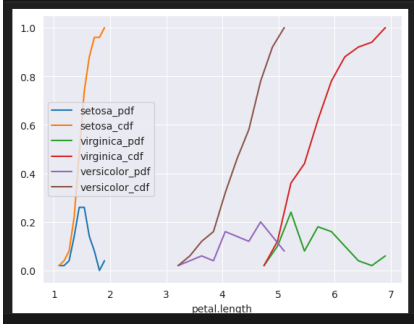
```
counts, bin_edges = np.histogram(iris_setosa["petal.length"], bins=10, density=True)
pdf = counts/sum(counts)
cdf = np.cumsum(pdf)
plt.plot(bin_edges[1:], pdf, label="setosa_pdf")
plt.plot(bin_edges[1:], cdf, label="setosa_cdf")

counts, bin_edges = np.histogram(iris_virginica["petal.length"], bins=10, density=True)
pdf = counts/sum(counts)
cdf = np.cumsum(pdf)
plt.plot(bin_edges[1:], pdf, label="virginica_pdf")
plt.plot(bin_edges[1:], cdf, label="virginica_cdf")

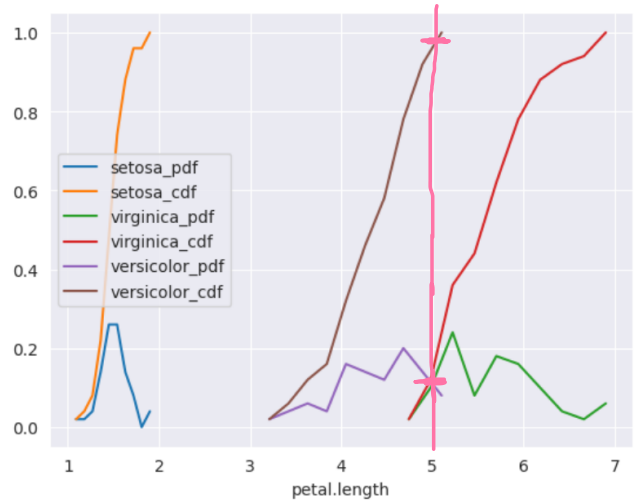
counts, bin_edges = np.histogram(iris_versicolor["petal.length"], bins=10, density=True)
pdf = counts/sum(counts)
cdf = np.cumsum(pdf)
plt.plot(bin_edges[1:], pdf, label="versicolor_pdf")
plt.plot(bin_edges[1:], cdf, label="versicolor_cdf")

plt.xlabel("petal.length")
plt.legend()
plt.show()
```

0.3s



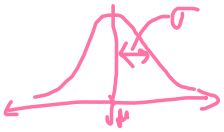

Consider the graph.
If petal.length == 5




---

The corresponding values of cdf for versicolor and virginica are approx 0.98 and 0.12
So 98% of versicolor will be labeled correctly if I choose versicolor
I'll make a mistake for 12% of virginica
So I'll choose versi.

Mean, Variance, Std. Dev



```
#Mean, Variance, Std-deviation,
print("Means:")
print(np.mean(iris_setosa["petal_length"]))
#Mean with an outlier.
print(np.mean(np.append(iris_setosa["petal_length"],50)));
print(np.mean(iris_virginica["petal_length"]))
print(np.mean(iris_versicolor["petal_length"]))

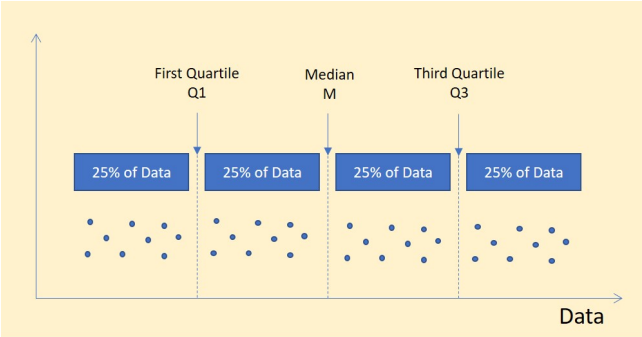
print("\nStd-dev:");
print(np.std(iris_setosa["petal_length"]))
print(np.std(iris_virginica["petal_length"]))
print(np.std(iris_versicolor["petal_length"]))
```

```
Means:
1.464
2.41568627451
5.552
4.26

Std-dev:
0.171767284429
0.546347874527
```

Mean and variance are highly affected by outliers

Median

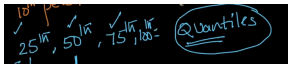



Median is the middle number in sorted data
So it isn't affected too much by outliers

Example:

Data = 1, 2, 3, 4, 5, 6, 7
Mean = 4, Median = 4

Say we add outlier = 100
Data = 1, 2, 3, 4, 5, 6, 7, 100
Mean = 64, Median = 4.5




```
print("\nMedians:")
print(np.median(iris_setosa["petal_length"]))
#Median with an outlier
print(np.median(np.append(iris_setosa["petal_length"],50)));
print(np.median(iris_virginica["petal_length"]))
print(np.median(iris_versicolor["petal_length"]))


print("\nQuantiles:")
print(np.percentile(iris_setosa["petal_length"],np.arange(0, 100, 25)))
print(np.percentile(iris_virginica["petal_length"],np.arange(0, 100, 25)))
print(np.percentile(iris_versicolor["petal_length"], np.arange(0, 100, 25)))

print("\n90th Percentiles:")
print(np.percentile(iris_setosa["petal_length"],90))
print(np.percentile(iris_virginica["petal_length"],90))
print(np.percentile(iris_versicolor["petal_length"], 90))

from statsmodels import robust
print ("\nMedian Absolute Deviation")
print(robust.mad(iris_setosa["petal_length"]))
print(robust.mad(iris_virginica["petal_length"]))
print(robust.mad(iris_versicolor["petal_length"]))
```

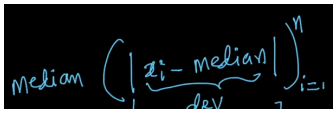
```
Medians:
1.5
1.5
5.55
4.35

Quantiles:
[ 1.     1.4    1.5    1.575]
[ 4.5    5.1    5.55   5.875]
[ 3.     4.     4.35   4.6 ]

90th Percentiles:
1.7
6.31
4.8

Median Absolute Deviation
0.148260221851
0.667170998128
0.518910776477
```

Median Absolute Deviation

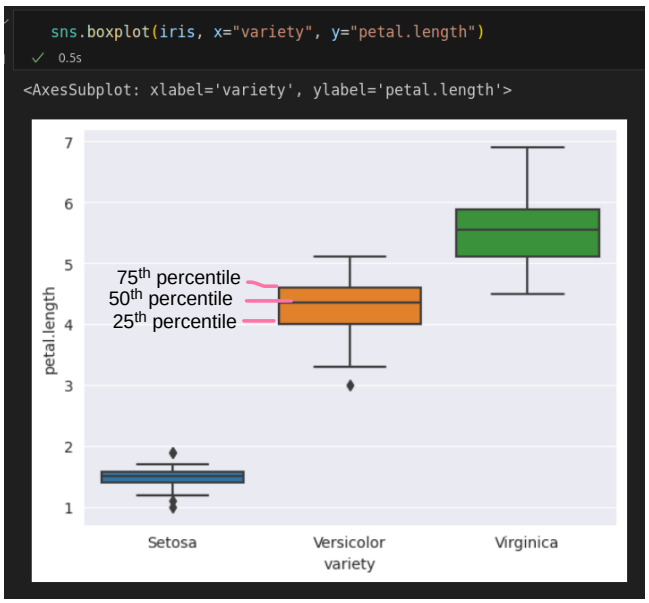





Kinda like std deviation
Consider the analogy:
Mean:median::stddev:MAD

Interquartile Range
$75^{th}$ percentile value – $25^{th}$ percentile value

Box Plots



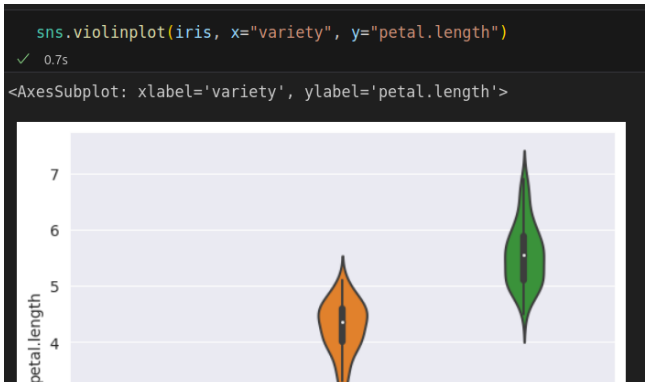

Helps visualize quartiles.
Width means nothing.

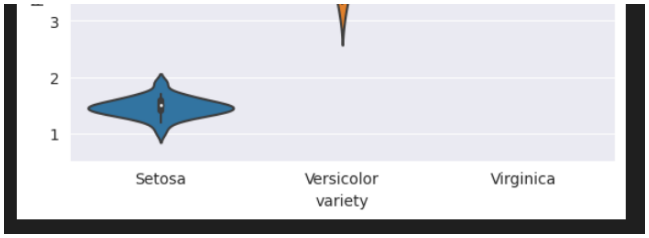


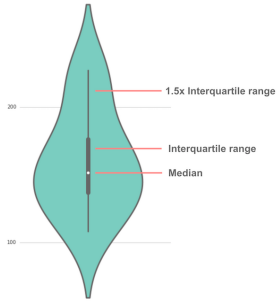

No standard way to draw whiskers. Seaborn here uses +-½*IQR length. Sometimes max and min values are used.
Most points lie within the whiskers.
The individual points are outliers.

Violin Plots

These combine prev 2 plots and simplify them.

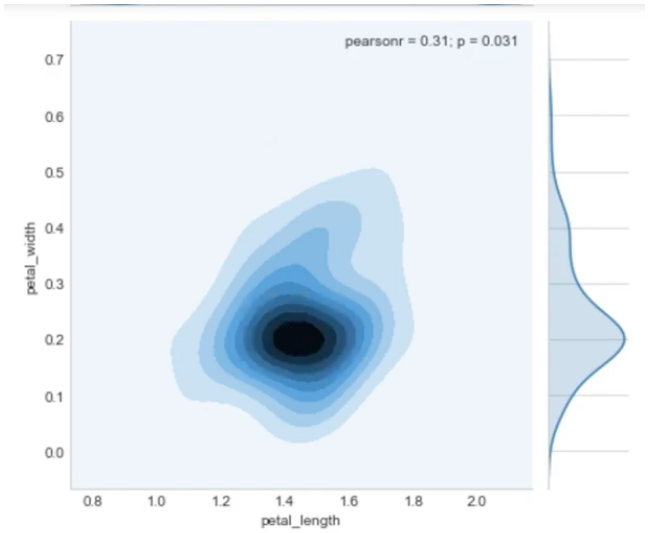





The prob density functions are plotted as well.

2d probability density plot
Contour plot

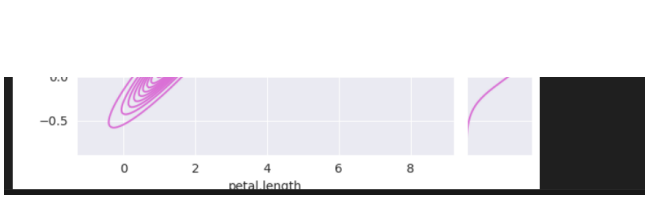



```
sns.jointplot(iris, x="petal.length", y="petal.width", kind="kde", color="orchid")
```

1.9s

```
<seaborn.axisgrid.JointGrid at 0x7f3c08887550>
```




0.0
−0.5
0
2
4
6
8
petal.length